Edge AI Fall Detection System: A Hardware vs. Software Architecture Comparison

Embedded Systems Mini-Project

1st Yossi Brim
Electrical and Electronics Engineering
Jerusalem College of Technology (JCT)
Jerusalem, Israel

2nd Elad Asbag
Electrical and Electronics Engineering
Jerusalem College of Technology (JCT)
Jerusalem, Israel

***Abstract*—Fall detection systems are critical for the safety of elderly individuals and dementia patients. While hardware-accelerated solutions utilizing Field-Programmable Gate Arrays (FPGAs) offer deterministic, real-time performance, they often entail high costs, complex memory management, and prolonged development cycles. This paper presents a software-based Edge Computing alternative using a Raspberry Pi and a pre-trained computer vision model (MobileNet-SSD). We propose a lightweight fall detection pipeline that processes video streams, extracts human bounding boxes, and communicates alert states via an I2C-based hardware display. The objective of this mini-project is to provide a Proof of Concept (PoC) and conduct a comparative analysis between the software-based Edge AI architecture and a traditional FPGA/RTL implementation in terms of development agility, hardware complexity, and overall efficiency.**

I. Introduction

The engineering challenge of detecting human falls in real-time is typically addressed using dedicated hardware systems to ensure low latency and high reliability. In our parallel senior design project, a fall detection system is implemented on an Artix-7 FPGA, leveraging custom RTL video pipelines, BRAM memory buffers, and bare-metal SCCB/VGA controllers. While this architecture provides uncompromising deterministic performance, it significantly increases the Time-to-Market (TTM) and engineering overhead.

In modern embedded systems, there is a growing paradigm shift towards Edge AI—running optimized machine learning models on accessible microcomputers. This project investigates the feasibility of replacing the complex hardware pipeline with a software-centric approach. By utilizing a Raspberry Pi and a pre-trained Object Detection deep learning model, we aim to extract spatial bounding boxes and apply geometric fall-detection logic in Python. Furthermore, to satisfy physical hardware interfacing requirements, the system acts as an I2C master bus, transmitting real-time alerts to a digital display. This paper explores the system architecture, the underlying AI algorithm, and evaluates the trade-offs of Edge Computing versus FPGA acceleration.

A. Architectural Comparison: Hardware (FPGA) vs. Software (Edge AI)

In developing the Edge AI software equivalent to the RTL-based Artix-7 implementation, two primary engineering challenges were addressed: resource management and temporal noise filtering.

1) Spatial Processing and Resource Optimization: In the FPGA environment, real-time video processing is heavily constrained by available Block RAM (BRAM) and DSP slices. Consequently, hardware implementations often require manual binarization thresholds and spatial downsampling to reduce high-resolution RGB data. Conversely, the software PoC abstracts this complexity using OpenCV's Deep Neural Network (DNN) module and a pre-trained MobileNet-SSD architecture. The framework automatically scales input matrices into low-resolution blobs (e.g., 300×300), allowing the Raspberry Pi 3 CPU to maintain a viable frame rate without explicit manual pixel decimation.

2) Temporal Noise Filtering (FSM Debouncing): Both implementations necessitate a mechanism to filter transient visual anomalies (e.g., a single distorted frame causing a false positive). In the VHDL implementation, this is managed via a synchronous counter within the Finite State Machine (FSM), asserting the detection flag only after the condition persists for a predefined threshold of clock cycles. The software PoC mirrors this exact logic utilizing a frame-based hysteresis threshold. The top-level controller (`main.py`) tracks consecutive frames yielding a positive geometric fall hypothesis. The hardware I2C display alert is triggered exclusively when `consecutive_falls` ≥ 2, ensuring high reliability and filtering brief temporal glitches.

II. Methodology and AI Algorithm

The proposed software-based Edge AI architecture shifts the computational paradigm from custom hardware logic to high-level algorithmic processing. The methodology is structured into four sequential stages: data acquisition, AI feature extraction, geometric evaluation, and state management.

A. Hardware Setup and Video Acquisition

The system utilizes a Raspberry Pi 3 equipped with a Camera Module 3 interfaced via the Camera Serial Interface (CSI). Unlike the hardware FPGA implementation which requires bare-metal SCCB configuration and strict pixel-by-pixel clock synchronization, the Edge PoC leverages the OpenCV library to abstract the video stream. This pipeline directly streams RGB matrices to the CPU, minimizing acquisition latency.

B. Edge AI Feature Extraction

To circumvent the need for custom convolutional accelerators and extensive BRAM utilization, the system integrates a pre-trained MobileNet-SSD model via the Caffe framework. Optimized for Edge devices, this specific architecture significantly reduces CPU load and inference time. The model processes the input frames, identifies objects, and isolates predictions corresponding to class ID 15 (”Person”). For valid detections with a confidence score exceeding 0.5 (50%), the system extracts the (X, Y) spatial coordinates of the enclosing bounding box.

C. Geometric Fall Detection Logic

Rather than deploying a computationally heavy action-recognition network, the system employs a deterministic geometric algorithm based on the aspect ratio of the extracted bounding box. Let W and H represent the width and height of the bounding box, respectively.

Under normal upright conditions, the subject's height inherently exceeds their width. A fall event is mathematically hypothesized when the subject's posture shifts horizontally, altering the bounding box geometry. To account for AI padding and varied physical proportions, an 80% safety margin is applied. The fall condition is defined as:

$$W > 0.8 \times H$$

D. State Management and System Output

To mirror the RTL Finite State Machine (FSM) implemented in the Artix-7, the Python top-level controller (`main.py`) executes a state machine with an integrated hysteresis filter. A localized fall hypothesis must persist for a minimum threshold of two consecutive frames to effectively filter temporal anomalies. Upon reaching this threshold, the system transitions to the `FALL_DETECTED` state and instructs the Hardware Abstraction Layer (HAL) to transmit an alert via the I2C bus to a 0.96-inch SSD1306 OLED display.

III. Results and Performance

Conclusion